The key concepts of Luthier and the logic that joins them

Contents

1. The thread **3**

2. A type is a theory, and its values are its models **4**
2.1 The idea . . . 4
2.2 The example: the theory of a tape . . . 4
2.3 Why the laws must characterize . . . 5
2.4 A model, and the claim . . . 5

3. Equality, the core, and its three planes **6**
3.1 Equality belongs to the object . . . 6
3.2 The levels of equivalence . . . 6
3.3 The three planes . . . 7
3.4 Properties said once, structures built from them . . . 7
3.5 The core as a prelude, and what may still move . . . 7

4. Names resolve by type **8**
4.1 The rule . . . 8
4.2 Why . . . 8
4.3 In the example . . . 9

5. Three kinds of file **9**

6. Testing a law: certifiers, generators, grades **9**
6.1 A law is not the procedure that checks it . . . 9
6.2 Generators are values . . . 10
6.3 A test program . . . 10
6.4 What a sample says . . . 11
6.5 The grade . . . 11

7. Why a law needs a control **11**

8. The certificate and its cone **12**
8.1 What a certificate records . . . 12
8.2 Why a certificate is keyed by a cone . . . 13
8.3 Frozen, superseded, and built only by certifiers . . . 13
8.4 Transport along identity . . . 13

9. Numbers and within-bound laws **14**
9.1 The ideal and its realization . . . 14
9.2 Every equality has a version within a bound . . . 14
9.3 The example . . . 14
9.4 How a budget scales with a bound . . . 15

10. Error, uncertainty, evidence and resources **15**

11. The world and actions **16**

12. The demand decides what runs **17**
12.1 Nothing runs until something outside asks 17
12.2 Why the demand decides 17
12.3 Certification is the default 18
12.4 The identity of a result 18

13. Specimens and the planner **18**

14. Claim, evidence and execution kept apart **19**

15. What is open **19**

Draft 0, 2026-10-21, written by Claude Opus 5.5 on the brief `record/plan/core-2026-10-17/prompt-concepts-doc.md`, for Luciano to annotate. Inputs: the repository at 01db6eb (`docs/principles.md` as projected from `record/principles-language.md`, `docs/spec/`, `docs/guide/00-fundamental-elements.md`, `record/decisions.md`, `record/plan/core-2026-10-17/execution.md` sections 0 to 4, and the library), and the example of `docs/concepts/` at e396fa1, which `luthier check` accepts and `luthier certify` certified on 2026-10-21 (seed 20261021). The PDF is built by the command in the comment at the top of this file.

This is written for a scientist who likes mathematics and writes some code, new to Luthier, who wants to know what its pieces are and why each one follows from the ones before it. It is neither a tutorial on the syntax nor the specification again: the rules are in `docs/spec/`, and where this text and the specification differ, the specification holds.

How claims are marked. Each statement cites where it comes from. A principle is cited by its number with its statement, as in "L10.12 (a store lives beside the file that claims what it certifies)"; the principles are `docs/principles.md`, organized under G0 to G2. A decision is cited by its id in `record/decisions.md` (D-2026-10-18-b), a clause of the specification by chapter and number (spec 12.S22), and a theory or model by its file. Four marks say how firm a statement is:

- [settled] marks a clause Luciano has marked, or a rule the interpreter implements;
- [proposed] marks a clause awaiting his mark (the principles mark these themselves);
- [building] marks what is decided and not yet built, or built in part, or suspended by a later decision; the core of the library is being rebuilt now (section 3.5 says what may still move);
- [mine] marks a reading or a proposal of this document, in no source.

An unmarked paragraph explains and adds nothing that its citations don't say.

Notation. Spec 1 fixes the symbols; these are the ones used below.

Written	Read
`x : T`	x is a T (for a type x, x refines T)
`x : T := t`	the definition of x, of type T, with body t
`x : T` with no body	a declaration: a name with a type and no body, a *hole*
`x = t`	an alias: another name for the term t
`U`, `W`	the universe (the type of types) and the world (section 11)
`A -> B`, `A * B`	functions from A to B, pairs of an A and a B
`{ x : A, y : B }`, `{ x := a, y := b }`	a record type and a record value; `e.x` its field x
`fn x { t }`	the function with parameter x and body t
`p & q`	the conjunction of two laws
`(name <- v)`	a named argument (an option of a call)

The subtype, the values of `T` for which the predicate `p` holds, is written `{ x : T | p }`.

In mathematical formulas, $\mathbb{R}$ is the real numbers, $\mathbb{Z}$ the integers, and $u = 2^{-53}$ the unit roundoff of IEEE 754 binary64 doubles, so that $\epsilon_{\text{mach}} = 2u = 2^{-52}$ is the distance from 1 to

the next double (spec 1.S9). The library calls ϵ_{mach} `epsilon`; spec 1.S11 keeps the letter ε for the tolerance a demand sets, and this text does the same.

1. The thread

The document follows one example from its first line to a run. The example is a tape measure: the instrument with which the first distances of sonolocalización were measured, between microphones and speakers on a floor (`examples/sonolocalizacion/A/graph`). Its four files are in `docs/concepts/`:

File	Kind	What it holds
`tape.lth`	theory	the type of a mark, the theory of a tape, its four laws
`tape.lut`	model	one tape, and the claim that its marks form a metric space
`tape.test.lut`	test program	the demand that certifies the model, with three controls
`tape.run.lut`	run	a demand on the world: will a cable reach?

The library's own claims of the same forms are core.lut's `IntIsAMetric` and numbers.lth's within-bound laws; they are spread over a dozen files and a long history, so the example restates their forms at the size of one page [mine: the choice of an example built for this document over one taken whole from the library].

The thread has ten steps, and each section below is one of them. Each step is there because the one before it leaves a question that only it answers.

1. A type is a theory, and declaring a value of it declares its tests (section 2).
2. The laws of a theory are stated over equality, and equality belongs to the object; the core gives the objects, their levels of equivalence and the structures, said once (section 3).
3. A structure said once is used by one name per concept, resolved by the type (section 4).
4. Theories, models and demands live in three kinds of file, so that the checker reads only statements and only a demand executes (section 5).
5. A law over an infinite domain can't be run whole; a certifier produces evidence of a grade (section 6).
6. Evidence that could not have failed is no evidence: every law carries a control (section 7).
7. The evidence is recorded in a certificate keyed by the hash of the cone it depends on, so that it is valid exactly as long as nothing it read has changed (section 8).
8. Numbers are realized by doubles, whose laws hold within bounds a theory states (section 9), and the quantities that measure "how far" form one algebra (section 10).
9. A program acts on the world only through actions, threaded by a token used once (section 11).
10. A demand from outside decides what runs, and refuses to run what isn't certified; one day a planner will choose among certified equivalents (sections 12 and 13).

Section 14 then shows how the three parts of the result, the claim, its evidence and its execution, stay apart, and section 15 lists what is still open.

2. A type is a theory, and its values are its models

2.1 The idea

In type theory a type is a proposition and a value of it a proof (G0 (type theory is the guide)). Luthier takes one step from there, Luciano's first eureka: if a type is a proposition, then "this structure satisfies the laws of the theory" can be said as "this structure is of the type of the theory". A value of that type is a *model* of the theory, and Luthier verifies that it is. So declaring `m : T` declares the tests `m` will pass, and coding and testing are one act (G1 (a type is a theory and its values are its models)) [settled].

The words are those of the Terms of `docs/principles.md` [settled]:

- a *structure* is a type built by the kernel's formers: products (records, tuples), sums, function types and recursive types, each possibly holding the others;
- a *law* is a predicate, a function into `Bool`, or a cost, a function into a type of costs (section 10), or a combination of laws;
- a *theory* is a structure with its law;
- a *model* is a value of a theory's structure whose law is certified.

In mathematical form: a theory is a pair (S, ℓ) of a structure S, usually a record of operations, and a law $\ell : S \to \text{Bool}$. Its models are the subtype $\{m : S \mid \ell(m)\}$, the equalizer of ℓ and the constant `true`, whose inclusion into S is the forgetful projection (L8.10 (the certified subtype from a law is the equalizer of the law and the constant true) [proposed]). The same construction, a material type and a law giving a new type, its only constructor and the function that says whether a value belongs, is the fundamental pattern of the language for values, functions and programs alike (L8.5 (from a material type and a law come the certified type, its only constructor and the forgetful projection); L8.6 (a law gives the three at once)) [settled].

Luciano's own framing (D-2026-10-12-a: Luthier is presented as a verifier of models of theories, not of functions): a function seldom stands alone. Every law is an equality, and the equality of a carrier is itself a function with laws that a new function must respect, so the common case is a theory that adds one symbol to types already modelled, and its model is a function in the ordinary sense (D-2026-10-12-e: an isolated function has sense only in a context) [settled].

2.2 The example: the theory of a tape

`docs/concepts/tape.lth`, without its header comment:

```
import "numbers.lth" Doubles absDistance

-- A mark on a fifty-metre tape, in millimetres from its zero. The bounds are the
 ↪  instrument's, part
-- of the type (7.S4), and keep every reading an integer below 2^53, so exact as a double
 ↪  (11.S5).
Mark : U := { k : Int | 0 <= k and k <= 50000 }

-- A tape is a way of reading the distance between two marks, in millimetres.
Tape : { reading : Mark -> Mark -> Float64 }

ReadingSymmetric (t : Tape) (a b : Mark) : Bool := t.reading a b == t.reading b a
InOrder : U := { a b c : Mark | a <= b and b <= c }
ReadingAdds (t : Tape) (p : InOrder) : Bool := t.reading p.a p.c == t.reading p.a p.b +
 ↪  t.reading p.b p.c
Step : U := { k : Mark | k < 50000 }
ReadingSteps (t : Tape) (a : Step) : Bool := t.reading a (a + 1) == 1.0

metres (k : Mark) : Float64 := toFloat64 k / 1000.0
MetresWithin (t : Tape) (d : Doubles) (a b : Mark) : Bool := {
  computed := absDistance (metres a) (metres b)
```

```
  exact := t.reading a b / 1000.0
  absDistance computed exact <= d.epsilon * (metres a + metres b + exact) }

Tape (t : Tape) := ReadingSymmetric t & ReadingAdds t & ReadingSteps t & MetresWithin t
```

Read in order:

- `Mark` is a subtype of `Int` carved out by a law, the pattern of 2.1. The bounds belong to the type because they belong to the instrument (spec 7.S4 (the bounds of a number are in its type)).
- `Tape : { reading : ... }` declares `Tape` at a record type and gives it no body. It is a hole: declared, not yet defined (L2 (declare, then define, then inhabit); L2.1 (a declaration without a body is a first-class hole, and nothing executes until it is defined)) [settled].
- The laws mention `Tape` while it is only a declaration, which spec 4.R1 (a declaration mentions declared names) allows. Each law is a predicate over a model `t`, its other parameters the values it quantifies over.
- The last line gives the declared name a body, the conjunction of its laws, and so defines the theory: the records `{ reading }` for which the four laws hold at every argument (spec 4.R6 (the body of a declared name is an inhabitant or a predicate)) [settled]. A conjunction of partially applied laws means each conjunct quantified over the variables it leaves missing (spec 7.S22, D-2026-10-18-f) [settled].

`MetresWithin` is a law of another kind, about the doubles; section 9 reads it. The three exact laws say what a tape reading is without saying how to compute it.

2.3 Why the laws must characterize

A law states a mathematical property of the operation it constrains. Checking a second implementation against the first shows only that the two are consistent, and the native definition is one more model checked against the law (L9.2 (a law states a mathematical property; a second implementation checked against the first is consistency)) [settled]; spec 4.R10 (a law characterizes; a second implementation is not a law) makes it a rule for whoever writes laws, and the checker reports a theory whose only laws compare implementations as consistent and not characterized (spec 4.S20) [building].

The tape's laws do characterize. For $a \le b$, `ReadingAdds` at $a = b = c$ gives $r(a,a) = 2\,r(a,a)$, so $r(a,a) = 0$; then `ReadingAdds` and `ReadingSteps` give $r(a,b) = b - a$ by induction on $b - a$, and `ReadingSymmetric` gives the case $a > b$. So the laws determine the reading, $r(a,b) = |a - b|$, and every model of `Tape` computes that function. Determination is the limit a definition may reach, and no definition has to reach it: laws accumulate, and a law need not determine its operation (L9.4 (laws accumulate, and a law need not determine the operation)) [settled]. The argument of uniqueness is written in the file's comment, as the pattern of the first theory files asks (guide section 8).

Why the insistence? A law that copies the implementation duplicates information and can't catch the error both copies share. A law that characterizes fails on any function other than the one meant, which is what makes its passing informative; section 7 turns this into a requirement.

2.4 A model, and the claim

`docs/concepts/tape.lut`:

```
import "tape.lth" Mark Tape

tape : Tape := { reading := fn a b { toFloat64 (if a <= b then b - a else a - b) } }

markIsAMetricModel : Metric Mark (structural Mark) := { d := tape.reading }
MarkIsAMetric (model : Metric Mark (structural Mark) <- { d := tape.reading }) : Bool :=
  Metric Mark (structural Mark) model

TapeModels := MarkIsAMetric
```

`tape : Tape := ...` is a definition whose type is a theory, so its value is a model and the laws of `Tape` are its obligations (guide section 4). The name passes through three states: declared, defined, certified, the last one law by law, each with a grade (spec 4.S6 (the three states of a name)) [settled]. Until `tape` is certified it is a value of `Tape`'s structure with a claim pending on it; section 8 says when the claim is met.

The next three lines are a second claim, on the core's `Metric` (section 3): the marks with this reading are a metric space. The claim is written as a law whose model is a defaulted parameter, so that a control can put a broken model in its place (section 7); this is the pattern of core.lut's `IntIsAMetric`. `TapeModels` gathers the file's claims, as numbers_model.lut's `NumberModels` does.

3. Equality, the core, and its three planes

3.1 Equality belongs to the object

Every law above is an equality, so before a law can be stated its type needs an equality. Luthier distinguishes four things: definition (`:=`), the identity proposition, decidable equality (`==`) and definitional equality by reduction (≡), and each object carries the observational equality that fits it (L3.1 (definition, identity, decidable equality and definitional equality are different things, and each object carries its own)) [settled]. `==` exists only where equality is structural, on `Int`, `Float64`, `String`, `Unit`, `Empty` and the products, sums, data and subtypes built from them, never on functions (spec 4.R8) [settled].

An *object* is a type with its equality, the base theory every other theory over it uses (Terms; D-2026-10-12-ak). In the core it is `Object`, in `lib/core/structures/equality.lth`:

```
Object (A : U) : { eq : A -> A -> Bool }
Object (A : U) (o : Object A) := Equivalence A o.eq
```

that is, a reflexive, symmetric and transitive relation, a setoid. `structural A` is the object whose `eq` is `==`, for the types that have one. Equality is a law of the object: Boolean where the carrier is decidable, a distance where it carries one, and the type of the identity law's result, `Bool` or a cost, says which (L3.3 [proposed]; spec 5.S4 (equality is a law of the object)) [settled].

3.2 The levels of equivalence

An equivalence answers yes or no. A metric answers how far. Luciano's reading of the core (D-2026-10-18-c, as L3.5 [proposed]) is that these are one structure over different bases: a category enriched over `Bool` with `and` gives an equivalence, over $[0, \infty]$ with $+$ a metric space, over distributions of distances with a t-norm a probabilistic metric space (Lawvere 1973; Menger 1942). Luciano took the three levels as a fundamental principle of Luthier, the one he finds for costs and probability (D-2026-10-18-h).

The metric level is in the core with its precise definition (D-2026-10-17-f: a metric space enters the core with its axioms; a floating-point realization is a separate claim within a bound) [settled]. A metric on a type A with its equality is a function $d : A \times A \to \mathbb{R}$ with

- (M1) $d(x, y) = 0$ if and only if $x = y$,
- (M2) $d(x, y) = d(y, x)$,
- (M3) $d(x, z) \leq d(x, y) + d(y, z)$,

non-negativity following from them. `equality.lth` states the three as laws over the type's `Object` (`MetricZeroOnEqual`, `MetricIndiscernible`, the core's `Commutative`, `Triangular`) and non-negativity as a fifth, stated for its control. The standard variants are there by their exact names: pseudometric, quasimetric, ultrametric, extended metric. A threshold on a metric gives a tolerance relation, reflexive and symmetric and not transitive; only an ultrametric gives an equivalence by a threshold (the correction to D-2026-10-18-c in `record/decisions.md`) [settled]. This last fact returns in section 13: it is why closeness can't be used as a class.

The tape's `MarkIsAMetric` is the claim that `(Mark, |a - b|)` is a model of `Metric`: exact, since the reading is an integer below 2^{53} and so exact as a double. Only distances computed without rounding are exact models of `Metric`; a distance computed with rounding is a model of `MetricWithin`, a metric within a bound (D-2026-10-17-f; section 9).

3.3 The three planes

D-2026-10-18-c names three planes in the core, and L3.5 states them [proposed]:

1. the objects (`Bool`, `Int`, `Float64`, `List`, functions, functors and the universal constructions: product, coproduct, equalizer, pullback and the rest, D-2026-10-17-g), each with its theory;
2. the levels of equivalence of 3.2, which are objects too, compared by refinement or by Lipschitz equivalence;
3. the computable types, models of the theories of the first two planes, certified against them.

The rule that matters: the theories of planes 1 and 2 only declare and state laws, with the mathematical references they are tested against; whatever computes lives in plane 3. The second version of the library failed by mixing them, helper functions written into the theories (608 of them, with their repetitions) (D-2026-10-18-c). In the example, `Tape`, `Mark` and the laws are plane 1, `Metric` is plane 2, and `tape` with its `fn a b { ... }` is plane 3.

The rule has a reason in what comes later [mine: the reading]. A theory that holds no computation can be read without running anything, so the checker can read theories freely and a demand reads a model only when it needs it (section 5); and a theory with no computation of its own can't become a second implementation of its own laws, which is the failure 2.3 warns against.

3.4 Properties said once, structures built from them

A property such as commutativity is written once, generic over the type, its equality and the operation, and every type claims it (D-2026-10-17-b: the algebraic properties are generic laws over an operation and its equality, said once; D-2026-10-17-c: their names are English adjectives, the type first) [settled]. From `lib/core/properties/operations.lth`:

```
Commutative (A B : U) (o : Object B) (op : A -> A -> B) (x y : A) : Bool := o.eq (op x y)
 ↪  (op y x)
```

A structure is the conjunction of its properties over its fields (`Monoid = Associative & Neutral`), written bare, since the tester splits a conjunction into its conjuncts (D-2026-10-18-f: no one-line wrapper per law and structure) [settled]. A type claims each structure it satisfies in the file that defines the type, so the file is the concept whole, and nothing is said twice: where the role is a theory stated elsewhere, the file claims that theory's model (L2.8 [proposed]). `Doubles` claims `Commutative` on `plus` and does not claim `Associative`, which documents by absence where the doubles are not the reals (D-2026-10-17-b).

Luciano's reason is a principle of style: Luthier is written in the manner of Haskell, a property said once, generic over the operation and its equality, each type claiming it; the same thing restated per type or per file is a defect of the library, and it is measured (L7.6 [proposed]; D-2026-10-18-a). The measures are the core's end, in `execution.md` section 0: no name defined at the top level of two files, no list helper reinvented per type, no Boolean law left unclaimed, the `loose` notices only where real, the suite green [building].

3.5 The core as a prelude, and what may still move

The whole of `lib/core/` is the prelude: every file under it is loaded before any other, and its names are visible in every file without an import (D-2026-10-18-b; spec 12.S4, 12.S22) [settled]. A file that defines a name the core defines is refused, naming the core file. Arrays, tables, probability and the rest are imported by name until analysed; some may become core.

The rebuild of the core is in progress [building]. On 2026-10-21 bites 1 to 8 of `record/plan/core-2026-10-17/execution.md` had landed, bites 9 and 10 (the machinery and Luthier's self-description) were committing, bite 11 (arrays and tables) was running beside this document, and bites 12 (probability, belief, optimization, plot, the examples) and 13 (closing) were to come. What may still move:

- the names of claims and of their models (`IntIsAMetric`, `intIsAMetricModel`) and the gathering of claims (`CoreData`, `NumberModels`, and so this example's `TapeModels`);
- `Doubles`, which bite 12 makes `Doubles (F : U)` claimed at `Float64`, so that `plus` and `times` resolve at the doubles (execution.md, status of 2026-10-21);
- the container structures of `structures/containers.lth` and the list library, which bites 7 to 11 are still migrating to;
- the claim form with a defaulted model, which no decision fixes and which the core uses today.

The concepts in this document don't depend on those names; the example's lines may need renaming when the bites land.

4. Names resolve by type

4.1 The rule

Once a structure is said once, its operations need names, and there are two ways to name them. One gives each type its own names (`lengthOf`, `listLength`, `mapList`), which is how the library had 110 functions reinvented per type (D-2026-10-17-d). The other, decided by D-2026-10-18-b, gives each concept one name: one `map` of `Functor`, one `size` and `at` of `Indexed`, one `d` of `Metric`, one `eq` of `Object`. Each use is resolved by the model the type names [settled].

Spec 4.R13 (an operator is resolved by the type, and a type names its model) states the rule [settled], extended from operators to every field of a structure: a field written unqualified is resolved by the type of its first argument whose type is headed by a parameter of the structure, "under instantiation and through a subtype to its base"; `S.f` names the structure when two structures at a type share a field name. A carrier with two models of one structure is two types, and the conversion between them is explicit (D-2026-10-11-j). A parameter typed by a structure and omitted at a call is filled by the model its types name; a different structure is passed by name (D-2026-10-18-g) [settled]. Failure is an error: no model, or two, at the type is refused, naming the field, the type and the models claimed.

4.2 Why

The model of a structure at a type is a fact about the type, recorded once where the type is defined (L2.8 (a definition and the law of its role live in one file, the one that defines it) [proposed]). If each use had to repeat it, the fact would be stated many times and could be stated differently each time; resolving by the type says it once. And since equality belongs to the object (3.1), a generic `member x xs` must use the equality of the type of `x`, which only the type can supply.

D-2026-10-18-g splits a generic call into three decisions with three owners: the structure of the type (the theory's, filled from the type's model when omitted), another structure (the author's, written by name), and how close "equal" is when the equality is graded (the demand's: a cutoff, never fixed by the type's model or by the resolution). "The demand fills parameters; it does not swap structures." This keeps the meaning of a program in its text and the tolerances in its demand (section 12).

Why failure must be an error: before the typing bite of the rebuild, an operator whose left operand had no inferred type fell back to the bare name, which is how `x + "a"` passed `check` (execution.md 2.2, rule 5). With one name per concept there is nothing to fall back to, and a silent fallback would leave a stuck term for a run to find.

4.3 In the example

`docs/concepts/tape.run.lut` asks `d microphone speaker`, with `microphone : Mark`. The name `d` is `Metric`'s field; its first argument has type `Mark`; the run imports `markIsAMetricModel : Metric` from `tape.lut`; so `d` resolves to the tape's reading. Typed with an argument of type `String`, `check` refuses it with rule 4.R13, listing the models of `Metric` claimed.

One finding while writing the example [mine]: without the import of `markIsAMetricModel`, `check` still accepts `d microphone speaker`, and the elaborator resolves `d` to core.lut's `IntIsAMetric`, which is claimed on `MetricInteger`, another subtype of `Int`. The resolution went through `Mark`'s base to a claim on a sibling subtype. Here the two readings agree, since `Mark` lies inside `MetricInteger`; in general a claim on one subtype of a base says nothing about another, so this may be a hole in "through a subtype to its base".

5. Three kinds of file

A program lives in files of three kinds, and the suffix declares which (spec 12.R2 (three kinds of file, declared by the suffix); D-2026-10-16-e; L2.6) [settled]:

- a theory, `.lth`: types, declarations and laws. The checker rejects an `extern`, an action or a model value in it (spec 12.S23);
- a model or library, `.lut`: definitions that claim a theory, each model in a file of its own, executed only when a demand reaches them;
- an executive file, which carries a demand with its entry and is never imported: a run, `x.run.lut`, with the `main` that `luthier run` applies to the world, or a test program, `x.test.lut`, with the `test` lines that `luthier certify` runs.

The separation came from C++'s headers and implementations, Luciano's proposal, with one difference: the drift between a header and its implementation that C++ leaves unchecked is caught, since each model is certified against its theory (D-2026-10-13-i) [settled]. An import names what enters with its theory, `import "x.lut" m : T`, so the importer is checked against `T` without reading `x.lut` (L6.14) [settled], and the importer sees the model only through its theory: privacy is ascription (L6.12 (an importer sees a module only through the theory it gives the module as type)) [settled].

The reason for the three kinds is laziness, and laziness is a consequence of the demand (section 12): a file is read when some demand needs it, an extern is a route opened only when a demand reaches its node, and a module is a value whose file is read when a demand first needs one of its names (L11.7 (a file is read when some demand needs it)) [settled]; spec 12.S21 [building], since the loader still reads imports at once (spec 8.S7). For this to be safe, reading a theory must never touch the world, which is what the refusals of 12.S23 guarantee. And a demand needs a place of its own, so that importing a library never runs a test or an action by accident: that place is the executive file.

Where results go follows the kinds. A run's records live beside it (`x.run.json`, `x.results.json`, spec 12.S24). Certificates never live beside an executive file: each entry goes to the store of the file that makes the claim (L10.12 (a store lives beside the file that claims what it certifies) [proposed]; spec 12.S10 [settled]). For the example that store is `docs/concepts/tape.cert.json`, beside `tape.lut`.

6. Testing a law: certifiers, generators, grades

6.1 A law is not the procedure that checks it

The laws of `Tape` quantify over every mark and every ordered triple of marks: $50001^3/6$ triples, about 2×10^{13}. A law over `Int` or `Float64` quantifies over infinitely many values. Running the law over its whole domain is out of the question for most laws worth writing. So Luthier separates the law, which is the proposition, from a *certifier*, a method that produces evidence for it; one law may have several (L9.1 (a law is the proposition; a certifier produces

evidence for it, and one law may have several) [proposed]; D-2026-10-12-k [settled]). The five certifiers are:

- proved, by a theorem cited or by a library's guarantee (spec 6.R1, 6.S9);
- exhaustive, over every case of a finite domain;
- cases, over chosen cases;
- sampled, over a sample drawn from a stated distribution;
- assumed, not checked (the laws of an `axiom`, spec 11.R3).

The certifier is chosen by what the law admits: what is known by a theorem is cited, what can be enumerated is checked exhaustively, what can only be sampled is checked statistically (L9.5) [settled]. The evidence follows the form of the law: a `forall` is covered exhaustively, by listed cases or by a sample, an `exists` by a witness, `and` by both sides and `or` by one (L9.11) [settled]. The two quantifiers are dual: a `forall` is refuted exactly by one counterexample and, over an infinite domain, confirmed only by statistics; an `exists` is confirmed by a witness (L9.12) [settled].

6.2 Generators are values

A sampled test needs values to draw, and drawing a value is a function: a generator is a value of type `Gen T`, a size and a seed giving a value and the next seed (spec 7.S1) [building: `Seed` in place of `Int` is decided]. No type owns a generator, so every test says which generators it used and its certificate records them (spec 7.R1, 7.R2) [settled]. Generators are functors over types: when a type is well defined its generator follows its structure, so the generic generators are few (exhaustive; sampled with a linear or a logarithmic placement; a census; a single value), and the bounds of a number are part of its type (D-2026-10-06-ab; spec 7.S3, 7.S4) [settled, the table of 7.S3 in part building]. A generator that claims a distribution states it as a law, certified as a statistical law is (spec 7.S1). This answers Luciano's observation on the old random library: it never defined the invariant of stochasticity, and a random library should define it and the tests of it, and use an extern, in batches, for speed (`record/Lucianos observations/random.lut.observations.txt`).

6.3 A test program

The test is a demand, so it lives in an executive file (spec 7.S21) [settled]. `test` is a function with named arguments: the mode, the number of cases, the size, the seed, the bindings and the budgets (spec 7.S20) [settled]. From `docs/concepts/tape.test.lut`:

```
models : List Binding := [
  { parameter := nothing, type := just "Tape", generator := "tape", coverage := Listed },
  { parameter := nothing, type := just "Doubles", generator := "doubles", coverage :=
   ↪  Listed } ]

test "Tape" (SampledMode linear) (cases <- 200) (size <- 50000) (seed <- 20261021)
 ↪  (profile <- models)
```

The profile binds the parameter of type `Tape` to a census of one value, the model `tape`, and the parameter of type `Doubles` to numbers_model.lut's `doubles` (spec 7.R1, 7.R4: a function over an infinite domain, or a value of a theory, is drawn from a census of models). The marks are drawn as integers in $[-50000, 50000]$ and filtered into `Mark` (spec 7.S3, 7.S6). The tester splits `Tape` into its four conjuncts and gives each its entry (spec 7.S22). `luthier certify docs/concepts/tape.test.lut` printed, on 2026-10-21:

```
docs/concepts/tape.test.lut: 2 laws, 3 controls, 5 lines ran, 0 known
  Tape: sampled; controlled (refuted on its control at ReadingAdds t: t = hooked, p = { a
   ↪  := 36254, b := 41964, c := 44418 })
  MarkIsAMetric: sampled; controlled (refuted on its control at MetricZeroOnEqual Mark
   ↪  (structural Mark) model: model = hookedMetric, p = { x := 0, y := 0 })
```

followed by the three control lines of section 7.

6.4 What a sample says

A sampled pass is a statistical claim about a deterministic law: a bound on the fraction of the distribution where the law fails, with the total number of cases drawn (D-2026-10-16-g: a sample of cases is a statistical law applied to a deterministic problem of infinitely many cases; L10.3) [settled]. In formula, if n cases were drawn and none failed, the fraction p of the distribution where the law fails satisfies, at confidence $1 - \delta$,

$$p \le \hat{\varepsilon}(n, \delta), \qquad \hat{\varepsilon}(n, 0.05) \approx \min(1, 3/n),$$

the rule of three (Hanley and Lippman-Hand 1983; spec 7.S11). For the 200 cases of each conjunct of `Tape`, $\hat{\varepsilon} = 0.015$: at 95% confidence, fewer than 1.5% of the drawn distribution's cases fail. The first instance of the conversion of that bound to an evidence cost is $-\log(1 - \hat{\varepsilon}(n, \delta))$ nats, with δ the demand's parameter (L10.3 [settled]; spec 7.S11 [building: suspended by D-2026-10-14-c, first everything Boolean]). Today an entry records the grade `Sampled` and its counts, not $\hat{\varepsilon}$.

Two refinements keep this honest.

- Sampling bounds the failure fraction under its distribution and says nothing of its null sets. The cases a law turns on come from three finite sources: the degenerate cases the type's theory declares, the two sides of each cutoff and the boundary of each premise, and each branch of the code; the evidence is kept per region (L9.16 [proposed]; D-2026-10-16-i, -j) [building: the first two sources run]. The example's entries record their regions: the parameter `p` of `ReadingAdds`, drawn from the subtype `InOrder`, has the boundary of that subtype as a region with its own count.
- A run under a new seed is a new stretch of the same sample, so the runs of one law, cone and binding pool into one sample of their total size (D-2026-10-16-g; spec 7.S13) [settled].

A statistical law, one whose correct model fails with some probability (a uniformity test, the transform of residuals), has a ternary verdict: it holds, fails, or is unresolved with what was sampled. The tester settles the grey zone by Wald's sequential probability ratio test, with the error rates α (rejecting a law that holds) and β (accepting one that fails) against an alternative at a stated distance, up to the demand's budget (L9.15; D-2026-10-15-b) [settled].

6.5 The grade

The grade of a certificate is one of the five certifiers' names, and it is derived from the domain and the evidence cost: proved and exhaustive have evidence cost 0 on their domain; cases, 0 on the cases listed and ∞ outside; sampled, the finite cost of its statistics; assumed, the cost the demand assigns to assumptions, with no default, so that a demand that names none refuses assumed values (L10.2 [proposed]; D-2026-10-12-r; spec 6.S7) [building]. The five names are for the reader; the planner reads the costs. When a law is a conjunction, how the grades of its parts combine is the demand's choice, the weakest by default (spec 5.S8) [settled].

7. Why a law needs a control

A passing test shows that the law held on the cases drawn. It shows nothing about the law unless the law could have failed. The external review of layer 3 found laws that could not: a host that does nothing passed `Host`, a score of zeros passed `IsGradient`, `phi - 1` passed `StepRule` (`record/IA/layer-3-review-2026-10-15/summary.md`). Green runs never show such a law; only a wrong model does.

So every law carries, with its evidence, a model it refutes: a negative control broken on purpose. A law that no broken specimen can fail describes nothing, and is a finding against the theory; a theory is done when each of its laws has its refuted model in the evidence (L9.14

(a law is evidence of nothing until a wrong model refutes it); D-2026-10-15-a) [settled]. `check -v` reports a law with no control as untested against any wrong model (spec 7.S19) [settled; the rate of detection over several seeds building]. In the vocabulary of statistics the control measures power: a law at level α that never rejects a wrong model has power 0 against it, and its passing is uninformative.

The example's controls, from `tape.test.lut`:

```
hooked : Tape := { reading := fn a b { tape.reading a b + 1.0 } }
test "Tape" (SampledMode linear) (cases <- 200) (size <- 50000) (seed <- 20261021)
 ↪  (profile <- onHooked) (control <- true)

noRoom : Doubles := { plus := doubles.plus, minus := doubles.minus, times := doubles.times,
  divide := doubles.divide, less := doubles.less, leq := doubles.leq, greater :=
   ↪  doubles.greater, geq := doubles.geq,
  unequal := doubles.unequal, epsilon := 0.0 }
test "MetresWithin" (SampledMode linear) (cases <- 200) (size <- 50000) (seed <- 20261021)
 ↪  (profile <- onNoRoom) (control <- true)
```

`hooked` is a tape read from the outside of its hook, one millimetre long at every reading, the mistake a person makes with a real tape. `ReadingAdds` refutes it at the triple the run reports ($a = 36254$, $b = 41964$, $c = 44418$: $8165 \neq 5711 + 2455 = 8166$), and `MetricZeroOnEqual` refutes its metric at $x = y = 0$. `noRoom` is the doubles with no room for rounding, which makes the bound of `MetresWithin` zero; the rounding of the division by a thousand refutes it at $a = 26150$, $b = 14398$. Each control line is the law's line with `(control <- true)` and a broken model in the profile; the tester records a refutation as a control's entry, never as the law's failure, and `check -v` then reports each law "controlled".

The last control shows that the bound does work: without it the law fails, so the law is not satisfied by the doubles for free. It doesn't show that the bound is tight. For that the library puts a control just below the bound: D-2026-10-17-f first wrote the doubles' associativity within $\gamma_2(|x| + |y| + |z|)$, the correction to it showed the bound needs the factor 3, and a control with a third of `epsilon`, which brings $3\gamma_2$ just below γ_2, is refuted at a listed case (numbers.lth §2; `tests/tester/numbers_controls.test.lut`). The example has no such control [mine: a gap in the example, left since its bound has room by design].

8. The certificate and its cone

8.1 What a certificate records

A certificate records what is known of a law: the law it records as checked, the domain the check covered (a subtype, the cases listed, or the distribution sampled), the bound observed, and the evidence, its grade and for a sample its statistics converted to an evidence cost (L10.1 [proposed]; spec 6.S6). Today, by the Boolean interim, an entry of the store holds the law's name, its outcome, the mode and count it ran with, the generators and their fingerprints, the seed, the cases evaluated, the budgets, the reach of each parameter, the regions, the grade, the interpreter's revision and the hash of the cone; the bound and the evidence cost are not recorded (spec 6.S6, [implemented] part) [building]. One entry of `docs/concepts/tape.cert.json`, its binding and budgets left out:

```
{ "law": "Tape", "conjunct": "ReadingAdds t",
  "cone": "sha256:2727dcc0bbc98f05a72820bcc138fd90a8431b6a4f67be1964fb689febdf5c30",
  "seed": 20261021, "cases": 200,
  "evidence": { "Points": { "evaluated": 200, "raised": 0, "coverage": "(Drawn 20261021)",
                            "verdict": "Supported" } },
  "grade": "Sampled", "outcome": "Passed", "interpreter": "v0.2",
  "regions": [ { "region": { "source": "FromLaw", "name": "p boundary" }, "n": 200 } ] }
```

8.2 Why a certificate is keyed by a cone

The *dependency cone* of a definition is the definition and everything it refers to, transitively. A certificate is identified by a hash over that cone in canonical form (bound variables renamed, module prefixes removed, the type annotations of binders erased), together with the host code behind each extern it calls and the interpreter, and with the binding that drew its cases (L10.4; spec 6.S8, 7.S9, 3.S34; D-2026-10-15-e) [settled]. In formula, for a law ℓ at a name n under a binding g (the generator that drew each parameter),

$$\mathrm{id}(\ell, n, g) = (\ell,\ H(\mathrm{canon}(\mathrm{cone}(n)) \,\|\, \mathrm{hosts} \,\|\, \mathrm{interpreter}),\ g),$$

with H the SHA-256 hash, canon the canonical form, hosts the host code behind the cone's externs, and $\|$ concatenation.

The reason is L4.7 (a definition's result depends only on its inputs) [settled]. A law's verdict on a name is then a function of the code the name reaches and of nothing else, so the cone is the exact set of things whose change could change the verdict. A key smaller than the cone (the name alone, the file) would accept an entry after a dependency changed; a key larger than it (the whole program, the repository) would void entries on changes that can't affect them. The canonical form is the strict level of equality, the hash (L3.4 [proposed]): two programs that differ only in the names of their bound variables, or in an annotation the elaborator writes, change nothing a term does and keep their entries. The binding is in the identity because a sampled certificate is evidence only about the distribution it drew from (L9.6 (the grade of a sampled certificate depends on the distribution sampled)) [settled]: "certified" is always "certified under a binding".

So an importer accepts an entry unchanged, with no new check, when it passed and its hash equals the current hash of the cone; an entry whose hash differs is stale and the obligation is pending again (spec 12.S11) [settled]. A check is made once, and belonging to a type is the record that it was (guide section 5, Luciano's 2026-10-06 note).

8.3 Frozen, superseded, and built only by certifiers

A certificate is a frozen snapshot: a later check doesn't change it, it issues a new one that supersedes it, both kept and the latest in front, so progress is the chain of certificates (L10.4; P3 (keep the original evidence and its corrections; put the best reading in front)) [settled]. An exact replay that agrees writes nothing; one that disagrees is kept beside the old and refuses the law until the disagreement is resolved, since something outside the identity changed the result (D-2026-10-15-f; spec 6.S8) [settled].

No operation on certificates raises a grade, lowers an evidence cost or widens a domain without new evidence (L10.9) [settled], and the language enforces it: only a certifier builds a certificate, the constructor of `Certificate` being in no interface outside the theory of certification, as only the kernel's rules build a theorem in the provers of the LCF family (Milner's Logic for Computable Functions) (L10.10) [settled]. Certifiers are values, and certification is no keyword; Luthier's shape is a triad, to say (`U`), to act (`W`) and to certify, and a certifier is the one action whose result re-enters the declarative face as a trusted value (D-2026-10-13-d); what it changes is the belief that a value belongs to a type (D-2026-10-13-e) [settled].

8.4 Transport along identity

If B is certified on inputs X with bound b and evidence cost e, and the identity of A and B is certified on inputs Y with distance d and evidence cost e′, then A is certified on $X \cap Y$ with bound $b + d$, evidence cost $e + e'$, at the weaker of the two grades, and on nothing else; a path of identities composes link by link (L10.7 [proposed]; spec 6.S11, 6.S12 [building: suspended by D-2026-10-14-c]). Two links sampled over the same distribution compose by the union bound, their failure fractions summed (L10.8 [proposed]; D-2026-10-12-q). Where the identity is Boolean, == structural, the certificate transports exactly (L9.9) [settled]. This is what will let the planner substitute one specimen for another (section 13) and what lets a renaming of a record's fields carry certificates at no cost (L3.4 (two types that differ only

in the names of their fields are equivalent by a certified renaming, along which certificates transport) [proposed]; D-2026-10-17-a).

9. Numbers and within-bound laws

9.1 The ideal and its realization

Code is written over ideal objects, the real numbers, and realized by concrete ones, the doubles; the realization's laws hold within a stated tolerance, and the error of each step is itself a law (L13.1 (code is written over ideal objects and realized by concrete ones)) [settled]. `Real` is the ideal, with its structure an axiom that no interpreter computes; every double is a real, so `Float64` refines `Real` in its values, and its operations are the machine's rounded ones (spec 11.R1, 11.S4) [settled]. The doubles are not a field: under cancellation no fixed relative tolerance makes addition associative, so `float64Field` doesn't exist (spec 11.S15) [settled].

Luciano reads rounding $\mathbb{R} \to$ `Float64` as a functor in Lawvere's sense, each law of `Real` projected onto `Float64` with its cost (L13.2 (his reading)) [settled]. It is non-expansive up to an additive defect of about $\epsilon_{\text{mach}}|x|$, since two reals astride a rounding boundary land one ulp (unit in the last place) apart however close they are, so the algebra of costs needs a multiplicative part where a Lipschitz constant exists and an additive part where rounding enters (L13.3, L13.4 [mark pending]).

9.2 Every equality has a version within a bound

D-2026-10-17-f [settled]: every property that is an equality has a version within a bound over a metric, the bound a function of the arguments taken from error analysis, never a constant (P11 (every criterion is a parameter, never a constant in the code)). A floating-point realization is not a model of the exact property; it is a separate claim. From `operations.lth`:

```
AssociativeWithin (A : U) (d : A -> A -> Float64) (op : A -> A -> A) (bound : A -> A -> A
↪  -> Float64) (x y z : A) : Bool :=
  d (op (op x y) z) (op x (op y z)) <= bound x y z
```

and numbers.lth claims it on the doubles' `plus` with Higham's bound,

$$|(x \oplus y) \oplus z - x \oplus (y \oplus z)| \le 3\,\gamma_2\,(|x| + |y| + |z|), \qquad \gamma_2 = \frac{2u}{1-2u},$$

with $\oplus$ the machine's rounded sum (Higham 2002, Lemma 3.1). D-2026-10-18-j [settled] states what such a law is about: a numerical law relates an input tolerance to an output tolerance, and that relation is its content. An agreement between a specimen and its definition becomes two laws with theorems' bounds, the algorithm's stability at exact inputs and the function's sensitivity to its inputs, so that the output's error is at most the sensitivity times the input's plus the algorithm's own; and every such law states as a premise its range of validity, since the bounds hold for small tolerances only (L9.18 [proposed]).

9.3 The example

`MetresWithin` is a stability law at exact inputs, of the simplest kind. The exact distance in metres is $D = |a - b|/1000$, with a, b the marks. The computed one divides each mark by a thousand, once rounded each, and subtracts, once rounded: $\hat{D} = |\mathrm{fl}(\mathrm{fl}(a/1000) - \mathrm{fl}(b/1000))|$, writing fl for the machine's rounding. With $A = a/1000$, $B = b/1000$ and the standard model $\mathrm{fl}(x) = x(1+\eta)$, $|\eta| \le u$,

$$|\hat{D} - D| \le u(|A| + |B|)(1+u) + 2u\,D \le \epsilon_{\text{mach}}\,(|A| + |B| + D).$$

The bound grows with $|A| + |B|$, the size of the marks, as well as with D: two marks far along the tape and close to each other lose relative precision in the subtraction. This is the

cancellation 11.S15 speaks of, at the scale of a tape. A demand that asked for that distance to within a relative 10^{-16} would ask for evidence the computation can't give: below the resolution of rounding a stricter cutoff buys nothing (L13.5 (a cutoff is read against the realization's defect at the point where it is met) [proposed]).

The law's range is the type of its variables, marks of the tape (D-2026-10-18-j), and its control is `noRoom` (section 7): with the bound at zero the law fails at a pair whose division rounds. L9.18 also asks the certificate to record how tight the bound is, the largest ratio of error to bound seen; today's entries don't record it [building].

9.4 How a budget scales with a bound

Luciano's candidate for a fundamental principle (D-2026-10-18-d; L9.17 [proposed]): the law of an algorithm states how its budget scales with the bound demanded, the resource (steps, samples, memory, bits of precision) as a function of the bound and of the problem's features (size, condition, unit roundoff), with the floor below which no budget suffices. It is the modulus of Bishop's constructive analysis and what LAPACK states in its error bounds and operation counts. The test of 6.4 is an instance: certifying a law by sampling to a failure fraction p at confidence $1 - \delta$ needs about $\ln(1/\delta)/p$ cases.

10. Error, uncertainty, evidence and resources

G2 (laws are valued in cost: one logic, extended to the non-negative reals) is Luciano's second eureka: Boolean type theory works for the integers and breaks on floating point, linear algebra and statistics, where a law holds up to an error; logic extended to $[0, \infty]$, with the Booleans as a special case, would let the composition of errors in numerical libraries be handled by one logic [settled as the organizer; its algebra open, O1].

What "cost" meant has since been split. D-2026-10-18-i [settled], written as L6.15 [proposed]: cost was four things.

- The *error*, a value's bound propagated operation by operation (intervals, balls, derivatives): the metric level.
- The *uncertainty*, a value as a distribution (Monte Carlo, the delta method, a pushforward): the probabilistic level.
- The *evidence*, what a certificate knows of a law with a finite budget: a failure fraction at most $\hat{\varepsilon}$ at confidence $1 - \delta$ (section 6.4).
- The *resources*, the budget (steps, samples, memory, time), scaling with the bound (9.4).

Error and uncertainty are data: decorations of the output, another model of the same theory over another carrier, computed in plane 3 (L5.4 [proposed]). A law propagates nothing. It is a Boolean claim at a bound about those quantities ("the distance to the reference is at most ε"), the tester measuring the quantity and the demand supplying ε. The four share an algebra: each is a model of a commutative ordered monoid, a quantale in its complete form, the base of Lawvere's enriched categories, and a function propagates them when it is non-expansive or Lipschitz, $d(f\,x, f\,y) \le c\; d(x, y)$ (D-2026-10-18-i). The bases:

Quantity	Carrier	Composition	Order
error	$[0, \infty]$	$+$	$\ge$
uncertainty	distribution functions of distances	a t-norm, or convolution	pointwise $\ge$
evidence	$[0, 1]$	$+$ (the union bound), or max when nested	$\le$
resources	$[0, \infty]$	$+$ in sequence, max in parallel	not stated

The order of the error base is Lawvere's, the opposite of the usual one, so that the empty

join is ∞ and the tensor preserves it (the correction to D-2026-10-18-i, from bite 6's finding 1); the decision names no order for the resources.

Three earlier facts still hold inside this picture.

- The lift is explicit: `lift : Bool -> Cost` sends true to 0 and false to ∞, `exact` reads a cost as `c == 0`, and `exact ∘ lift = id`, so `Bool` is a retract of `Cost`; nothing is promoted silently (L12.2) [settled]. Luciano: "no podes promover booleans a cost a troche y moche" (G2's record).
- The cutoff does not compose: two links each within ε are within 2ε, so the cutoff lives in the demand and is passed down from it, and no law carries one (L12.3 [proposed]; spec 5.S7) [settled].
- A probability in a certificate is that of a counterexample not yet seen; Bayesian evidence, a likelihood, is a density of any value, whose logarithm is no cost in $[0, \infty]$ (L12.6 [proposed]; D-2026-10-13-n) [settled].

Today all of this waits. First everything Boolean: every law is a Boolean predicate, and every cutoff it needs is a parameter of the law that the demand supplies, never a constant in a file; the cost types come later, on the failures of the sonolocalización slice (D-2026-10-14-c) [settled as the interim]. The core implements the Boolean and the metric bases; the probabilistic one is declared in `equality.lth` (Menger's axioms, no model claimed) and comes after, on the sequential test (D-2026-10-18-c) [building].

11. The world and actions

Every certificate so far leaned on one assumption: a definition's result depends only on its inputs, and effects live only in actions. That is the one assumption under which every evaluation strategy gives the same result (L4.7) [settled]. So a program is a graph of definitions with no effects, a DAG (directed acyclic graph) in which the dependencies give the only order there is (L4, L4.1) [settled], and the effects have to live somewhere the graph can see.

That place is the world. `W` is a primitive of the core beside `U`: definitions live in `U`, timeless, copied, compared and memoized; actions act on `W`. A program never constructs a `W`; the token enters at `main` or in the session, has no `==`, and each value of it is used once, as in Clean's uniqueness types (D-2026-10-13-b; L4.11) [settled]. An action is a function that takes the token and gives the next token with a result, `W -> ... -> W * A`, and a function whose type mentions no `W` has no effect (spec 10.S3, 10.S4) [settled]. Since each token is used once and two readings of one place before and after a write are two terms with a dependency between them, the order of actions is an edge of the graph like any other. There is one world; its parts, a file or the terminal, exist in Luthier's representation of it, and whether two are independent is a theory in a library (D-2026-10-13-a; L4.12) [settled].

An action is verified too, by observing the world before and after it, where the world can be read back; what can't be read back is typed, logged and audited (L4.9, L4.10) [settled]. Where no feedback exists (the system's delivery to the terminal, the file system, the interpreter, the floating-point hardware), there is a root of trust, declared once per project and listed in every run's record (L11.9 [proposed]; D-2026-10-16-n). An action on Luthier's own state, writing a certificate, is idempotent and memoizable; an action on the world never is (L4.11; spec 10.S6) [settled].

`docs/concepts/tape.run.lut`:

```
import "world_model.lut" HostConsole host : HostConsole
import "tape.lth" Mark
import "tape.lut" markIsAMetricModel : Metric

microphone : Mark
speaker : Mark
-- the cable's length in metres
cable : Float64
```

```
main (w : W) : W * Unit :=
  host.print w (if d microphone speaker <= 1000.0 * cable then "the cable reaches" else
   ↪  "the cable is short")
```

`host.print` is an extern route to the host's printing, and `world.lth` states what it does to the representation of the world (`Console`), with the host's actions certified against that description along one chain of tokens. A host action is validated at its first application in a run with the program's own arguments, and refused if the host disagrees (spec 10.S10) [settled]. An extern is declared over a type with laws and validated against them before it is applied; without a type with a law it is not applied (L11.2: "si el tipo no sigue la ley no se aplica el calculo") [settled]. A `provisional` extern is the declared exception, recorded as debt and refused by default (D-2026-10-16-c) [settled].

12. The demand decides what runs

12.1 Nothing runs until something outside asks

Type theory normalizes a term it already has. Luthier has no term to normalize until something outside the program asks for a value: a print, a file saved, a test. What governs execution is the demand, read in the same theory: a demanded type is a judgment to inhabit, and execution searches for an inhabitant (G0) [settled]. Luciano: "la compilacion se dispara por la demanda externa (es decir un print o un valor que sale al mundo), y la demanda determina el tipo que quiere el print" (`record/Lucianos ideas/principios organizadores.txt`).

The two faces of a program follow: the declarative face states what is known, universal and minimally processed; the executive face is a demand from outside, and the demanded type fixes the path, the types and the tolerances actually produced. The declarative face is the theories and the models; the executive face an executive file or a demand typed at the terminal (L5.1) [settled].

12.2 Why the demand decides

Because definitions have no effects (11), nothing in the declarative face has to happen; whatever happens is asked for. What is produced, and which types, is what the demanded value needs; the complement of its dependency cone is not produced, so the choice is canonical, one per demand (L5.13) [settled]. The default strategy plans backward from the demand over names, evaluating nothing, then runs the live subgraph strictly forward, each node once and whole (L5.2; spec 8.S3) [settled as decided; building: the backward pass is a hole in `lib/plan.lth`, and today `run` evaluates by call by need, which computes the same live subgraph without a plan as a value, spec 8.S4]. The same cone bounds what must be certified: what never reaches an output needs no certificate, and what reaches one carries its cone's (L10.6) [settled]. And it bounds what is read: the laziness of files in section 5 is this rule applied to files.

The demand also carries what the program leaves open. A demand is the gate through which parameters change what the program does: it carries its entry, the type demanded, the cutoff on the bound and on the evidence, the confidence δ for sampled evidence, the evidence cost of assumptions (no default), limits on resources, and the values of the program's holes (spec 8.S1) [building: the entry, the cutoff as a grade, the holes, the strategy and the limits run today, recorded whole in the run's record]. Every threshold or level is declared, passed and recorded (L5.18 [proposed]); the outermost demand carries its tolerance as a declared default, so even the root is a parameter (L5.19) [settled]. The tower of levels a theory generates may be infinite, and a computation walks only what its demand reaches (L5.21 [proposed]; D-2026-10-18-c: "La torre se corta en la demanda").

12.3 Certification is the default

Demanding a value demands every certificate in its dependency cone, paid once through the memo, with no opt-out at the start (L10.6; D-2026-10-11-n; D-2026-10-12-y) [settled]. `luthier run` refuses to start when any name in the cone of the entry is uncertified or provisional, listing each with its state (spec 12.S17) [settled]. A certificate serves every demand whose cutoff it meets, checked at the hit (L10.5) [settled; in the Boolean slice the cutoff is the weakest grade accepted]. No cutoff accepts a failure; running past a refuted law is said by naming it, `--accept assumed:LAW`, recorded and warned at every run (D-2026-10-16-h; spec 6.S15) [settled].

For the example, run as

```
luthier run docs/concepts/tape.run.lut --set microphone=1200 --set speaker=3450 --set
↪  cable=2.5
```

the cone of `main` holds, among others, `markIsAMetricModel`, `tape`, `Mark`, the core's `Metric` and the world model's `host`; `run` serves each from its store when its entry passed at the current hash and refuses otherwise. The holes are filled by `--set`, typed against their declared types (spec 12.S18). This document doesn't show the run's output: the rules of this repository leave runs to Luciano (`CLAUDE.md`), and the example was checked and certified, not run.

12.4 The identity of a result

A result is identified by its code, inputs, mode, demanded type and tolerance, the model its type names, the seed, the plan that produced it, and the hashes of the certificates in its live cone, so a changed host library or platform is a new result (L5.6) [settled]. `run` computes that key before evaluating and serves a pure entry, one whose cone holds no `W`, from `x.results.json` without evaluating; a cone that acts always executes (spec 8.S8, 12.S18; D-2026-10-15-d) [settled]. The meaning of a program, its realization under a plan and its evidence are three objects, never one (L5.10) [settled]: section 14 returns to them.

13. Specimens and the planner

A function has many implementations: fast or precise, in memory or on disk, native or borrowed from Python. Each is a *specimen*, a model of the function's theory certified against its laws, and the planner is what will choose among them by the performance the demander asks for (L6 (one function, many specimens, chosen by the performance the demander chooses, along certified identities); spec 9.S1) [settled as the principle].

The planner may substitute one specimen for another only along a certified identity between the two (D-2026-10-11-m) [settled]. The reason is in 3.2. In the theory, the specimens of a function form a class, the quotient by "computes the same function up to the law's equality". In a model that carries a distance, closeness is not transitive, so there is no class: if A and B are each within ε of S, A and B may be 2ε apart. There is only a graph of certified substitutions, and a substitution is a path whose summed distance and summed evidence are within the demand's cutoff; two specimens joined through a third may each replace it and not each other, and a lone specimen is never replaced (L6.5, L6.6 [proposed]; spec 9.S5 to 9.S7). On a decidable carrier the quotient is exact and there is no precision to trade; on a carrier with a distance, precision can be traded for resources up to the demand's tolerance (L6.4) [settled].

What the planner ranks is an efficiency, $\exp(-\text{cost})$ over the resources used, so that a specimen of cost zero has efficiency $1/\text{resources}$; with the demand's weight w this is the same as minimizing $\text{cost} + w \log(\text{resources})$ (L6.3 (his formula) [candidate]; spec 9.S15). The weight is the demand's, never the planner's (L6.2) [settled].

Today nothing chooses: a name is bound once and evaluation uses that binding (spec 9.S2) [settled]. The planner waits for certificates of identity, which wait for the cost types: after `v0.2`, execution before evidence, then evidence, then the demand and the planner (D-2026-10-13-ad) [building: spec 9.S5 to 9.S17 suspended by it]. For the example, `tape` is a lone

specimen of `Tape`; a second, say a lookup table read from a calibration file, would enter the planner's graph only by an identity certificate with its domain, and be certified on that domain and nowhere else (the example of L10: a lookup table equal to the native on a grid is certified on the grid and nowhere else).

14. Claim, evidence and execution kept apart

Three things make the language: what is said (the definitions, in `U`), what is done (the actions, on `W`), and what is certified, the action whose result comes back to the declarative face as the belief that a value belongs to a type, with its evidence (Terms; D-2026-10-13-d, -e) [settled]. L5.10 names the three objects of a result in the same spirit: the meaning, which the laws fix; the realization, the result under a plan; the evidence, its certificates [settled].

The three planes of section 3.3 and the three kinds of file of section 5 line up with these, in the table below [mine: the reading; no source states the correspondence].

	The claim	The evidence	The execution
What it is	a theory's laws and a model's declaration `m : T`	the entries that a certifier wrote	a run of a demand
Plane	1 and 2 state; 3 is the model claimed	about plane 3, measured against 1 and 2	plane 3, on `W`
File	`.lth` and the `.lut` that claims	the store beside the claiming `.lut` (L10.12, a store beside the claim)	the `.run.lut` and its records
Keyed by	the names and the theory	the law, the cone's hash, the binding	the result's key (L5.6, everything that made it)
Changes when	Luciano or an author rewrites a law	the cone or the binding changes	the demand or anything in the key changes

Each column can change without the others moving. A run doesn't change a certificate; a new test doesn't change the theory; a rewritten law changes the hash of every cone that reaches it, so the stale entries are seen as stale and the runs that rested on them can't be served. The separation is what makes "a run I can trust" a checkable statement: the run's record lists the certificates its cone was served by, each certificate names the law, the domain and the binding it holds for, and each law can be read without running anything.

In the example: `tape.lth` and `tape.lut` are the claim, `tape.cert.json` the evidence, and `tape.run.lut` with the `tape.run.json` that a run would write beside it the execution.

15. What is open

The questions of `docs/principles.md` that the concepts above rest on, each with what it blocks:

- O1, the algebra of costs: the conversion of a sample's statistics to a cost, the composition along a path, the four bases of section 10 as models of one structure. It blocks the cost types, transport (8.4), the planner (13) and the decorated outputs. D-2026-10-18-i proposes its shape; `Cost` is a library, its algebra decided when it is implemented (D-2026-10-12-al).
- O2, the type with its equality as the base of every theory, and how a type names its equality: answered in part by `Object` and resolution by type (sections 3 and 4), a second

equality on one carrier being another type (spec 4.R13) [mine: the reading that these answer O2 in part].

- O3, the world as a value and an opt-out of certification if a real case asks for one: the token is implemented; the stores are not yet places of the world (spec 10.S6, D-2026-10-15-d).
- O5, the decorated output: the derivative by dual numbers, the error by intervals, as other models of the theory the code was written over (L5.4 [proposed]).
- The three planes and the three levels of equivalence are in the principles as proposed clauses (L3.5 (the core has three planes); L5.21 (the tower of levels is cut by the demand)) and in the core's files; the specification has no clause for them yet.
- The core's rebuild (section 3.5): names may move until bite 13 closes, against the measures of `execution.md` section 0.